IN 213 **Énoncé**

OCamllex — Réalisation d'un analyseur lexical

Préliminaires

Dans ce TP, nous utiliserons OCaml, OCamllex (le générateur d'analyseurs lexicaux d'OCaml). Il est souhaitable que vous mettiez dans vos *bookmarks* la page contenant la documentation d'OCaml :

`https://ocaml.org/docs`

1 Comptage de lignes, mots et caractères dans un fichier

Q1 Réalisez à l'aide d'`ocamllex` un programme complet qui reçoit un ou plusieurs fichiers en argument, et qui affiche pour chacun d'entre eux le nombre de caractères qu'il contient. Afin de pouvoir utiliser le programme principal fourni en substrat, il convient que la règle de votre analyseur s'appelle `count` et le fichier `.mll` se nomme `cptlex.mll`.

Le programme principal (fichier `compte.ml`) pourra être le suivant, et devra être complété par le fichier `cptlex.mll` généré par `ocamllex`. Un fichier `Makefile` est aussi fourni. Installez ces deux fichiers dans votre répertoire de travail.

Le contenu du programme principal vous est donné ci-dessous, pour information.

Attention, vous remarquerez que la fonction `count` utilisée dans ce programme principal (fonction issue de la compilation de la règle nommée `count` dans le fichier `.mll`) prend 3 arguments et retourne 3 résultats (les nombres demandés en question 1, 2 et 3). C'est la version pour cet exercice complet (jusqu'à la question 3). Pour les questions 1 et 2, votre analyseur `count` (dans le `.mll`) retournera 0 pour les nombre non encore demandés.

```
(* Programme principal.
 * Ne fait que compter les caracteres.
 * Utilise une fonction count appartenant a un module Cptlex que vous
 * definirez par un analyseur count dans un fichier cptlex.mll.
 *)

let count filename =
  try
    let input = open_in filename in
    let lexbuf = Lexing.from_channel input in
    let (nbchars, nbwords, nblines) = Cptlex.count 0 0 0 lexbuf in
    Printf.printf
      "%s: %d caractere(s), %d mot(s), %d lignes(s)\n%!"
      filename nbchars nbwords nblines ;
    close_in input
  with
  | Sys_error msg →
      Printf.eprintf "Impossible d'ouvrir \"%s\" (%s)\n%!" filename msg
  | err →
```

```
    Printf.eprintf "Exception inattendue %s\n%!" (Printexc.to_string err)
  ;;

let main() =
  for i = 1 to (Array.length Sys.argv) - 1 do
    count Sys.argv.(i)
  done ;
  exit 0
;;

if !Sys.interactive then () else main () ;;
```

Attention : les analyseurs produits ne vont s'appliquer qu'*une seule fois* à l'entrée : quand on veut qu'un analyseur traite l'entrée complète, il faut :

— l'appeler itérativement si nécessaire (entrypoint(lexbuf), où lexbuf est la variable qui représente le buffer effectif)

— prévoir le cas **eof** (expression régulière prédéfinie correspondant à la fin de fichier) qui sera le cas d'arrêt normal.

Q2 Modifiez votre programme pour qu'il affiche le nombre de lignes des fichiers. On considèrera, comme l'outil `wc` d'Unix, qu'une ligne est une suite de caractères terminée par un retour à la ligne (`'\n'`). Ainsi, un fichier contenant uniquement une « chaîne » sans retour à la ligne final comportera 0 ligne. Attention, la majorité des éditeurs de texte, conformément à une convention Unix, rajoutent systématiquement un retour à la ligne final aux fichiers texte s'ils n'en contiennent pas.

Q3 Idem pour le nombre de mots.

2 Expressions arithmétiques

Q4 Définissez un type lexeme pour les éléments lexicaux d'un langage d'expressions arithmétiques composé d'entiers non signés, des 4 opérateurs classiques, de parenthèses, et de variables. Incluez un lexème nommé EOF qui correspondra à la fin de l'entrée (`^D` dans l'entrée standard), ou la fin de fichier ou de chaîne. Ce lexème sera produit lorsque l'expression rationnelle spéciale **eof** sera reconnue par l'analyseur.

Q5 Écrivez un analyseur pour ce langage. Naturellement, les espaces et retours à la ligne devront être éliminés au cours de l'analyse.

Q6 Augmentez votre analyseur pour qu'il puisse éliminer les commentaires à la C++ (`//` jusqu'à la fin de la ligne courante).

Q7 Ajoutez des commentaires multilignes de C (`/*` ... `*/`).

Q7 Ajoutez un ou deux mots-clés (par exemple une construction **let** ... **in** ...).